ESOF 322

Software Engineering

LECTURE: 4

J. L. Pach

OUTLINE:

- GIT stash
- GIT worktrees

GIT COMMANDS

GIT COMMANDS

- merge
- diff
- stash

WHAT IS .GITIGNORE?

The `.gitignore` file tells Git which files or directories to ignore — meaning they won't be tracked, staged, or committed to the repository.

This is useful for:

- Temporary files (e.g., .log, .tmp)
- Build artifacts (e.g., bin/, obj/)
- IDE-specific files (e.g., .vscode/, .DS_Store)
- Secrets or configuration files (e.g., .env, config.local.json)

HOW IT WORKS - .GITIGNORE?

You create a file named `.gitignore` in the root of your repository and list patterns for files or folders you want Git to skip. Example:

```
*.log
*.tmp
node_modules/
.env
```

- Git will ignore any file or folder that matches these patterns — even if they exist in your working directory.
- Use `.gitignore` to keep your repository clean and focused only on the files that matter. It helps avoid accidentally committing sensitive data or unnecessary clutter.

UNDERSTANDING .GITIGNORE – FILE, NOT A FOLDER

It's important to know that `.gitignore` is a **file**, not a folder. The naming convention comes from Linux/Unix systems, where files that start with a dot `.` are treated as hidden or configuration files. From a Windows perspective, `.gitignore` may appear as a file without a name and with an extension, or simply as a special file with no extension, starting with a dot. This can be confusing at first, but it's a common pattern in many development environments.

Examples of similar hidden/config files in Linux:

- `.ssh/` – stores SSH keys and config
- `.vscode/` – stores VS Code workspace settings

These dot-prefixed files and folders are used to configure tools and environments without cluttering the main workspace.

WHAT IS A GIT BRANCH?

A branch in Git is like a separate line of development. It allows you to work on new features, bug fixes, or experiments without affecting the main codebase. The default branch is usually called main or master.

WHAT IS A GIT BRANCH?

- Branches help teams collaborate safely and efficiently by isolating changes until they’re ready to be merged.
- Think of branches as parallel timelines. You can develop safely in one branch, test your changes, and merge them back when everything works. This is a core concept in modern version control.

GIT BRANCH – CREATE A NEW BRANCH

- The command git branch shows a list of all branches in your repository. The currently active branch is marked with an asterisk (*) `git branch new_branch`
- When working with a repository that has multiple branches, we can switch between them using two different commands. This is because modern versions of Git introduced standardized naming conventions, but the older commands were kept to ensure backward compatibility and to avoid forcing experienced users to relearn everything from scratch.

  `$ git switch second_branch` or `$ git checkout second_branch`

GIT STATUS & GIT BRANCH

```
git branch
* master
  second
```

```
git status
On branch master
...
```

Depending on the shell or terminal, Git can display additional information in the prompt, such as the current branch, whether all files are tracked, or if there are uncommitted changes. However, to check which branch you are on, you can always use the git status command or git branch without any parameters.

GIT MERGE

- A merge combines changes from one branch into another.
- **Typically, we merge into the main/master branch.**
- Example workflow:
  - Switch/Checkout main
  - Run git merge feature-branch
  - main now includes the changes from feature-branch

GIT MERGE

A merge creates a merge commit that keeps both branch histories visible. Useful when you want a complete history of how branches diverged and then rejoined.

Run from main

```
A---B---C---D  (main)
     \
      E---F (feature)
```

```
A---B---C---D---M  (main)
     \     /
      E---F
```

WORKING

with Branches

WORKING WITH BRANCHES

Branch inheritance:

- A newly created branch (other than main/master) inherits all files and commits that existed at the moment of its creation.
- In other words: it's a copy of the project's state at that time.

Files created inside a branch:

- New files committed inside a branch "live" only in that branch.
- The main branch (main/master) does not see these files unless a merge happens.

WORKING WITH BRANCHES

Merge command:

- The git merge <branch> command merges the given branch into the currently active branch.
- **Important**: the merge is always one-way – from the branch given as a parameter → into the branch you are on.

WHAT HAPPENS TO THE MERGED BRANCH?

- After a merge, the merged branch is not deleted automatically.
- Whether you remove it depends on team strategy and workflow:
  - Deleting keeps the repository clean and avoids clutter.
  - Keeping old branches may obscure the project and cause confusion.
- Remember: deleting a branch is **irreversible** if its changes are not merged.

WORKING WITH BRANCHES

Merge conflicts:

- A merge conflict happens when the same file was modified differently in two branches.
- Conflicts are very common in collaborative projects.
- Expect them – don’t assume merging will always be automatic..

RESOLVING MERGE CONFLICTS

- Git will stop and inform you which files are in conflict.
- Inside the file you'll see conflict markers like:
- The developer must manually decide which parts to keep.
- After editing and saving, run:
- to finalize the merge.

```
<<<<<<< HEAD
your current branch content
=======
incoming branch content
>>>>>>> feature-branch
```

`git add <file>` and `git commit`

SUMMARY 1/3

Let’s review what we have learned so far. We previously worked on creating new branches in Git and switching between them. One important observation is that files seem to be “hidden” when moving between branches, since each branch has its own version of the file system state.

- A newly created branch (other than main or master) inherits everything that existed at the moment it was created.
- Any new files added and committed in this branch “live” only in that branch. The main branch does not automatically have access to them.
- The merge command allows us to bring another branch into the currently active branch (never the other way around!).
- After merging, the absorbed branch is not automatically deleted.

SUMMARY 2/3

It is important to remember that merging cannot be **undone** easily, and the consequences are permanent. On the other hand, in large projects, a huge number of branches may appear to allow independent development of features. This branching helps with safety and organization, but leaving unsupported or abandoned branches can create confusion and reduce clarity in the codebase.

- ⚒ Merge conflicts are very common. They occur when two branches contain different versions of the same file. Hoping for no conflicts is unrealistic.

SUMMARY 3/3

- When a conflict happens, Git will report it and show which files are affected. Inside the file, Git will mark the conflicting sections with `<<<<<< HEAD`, `======`, and `>>>>>>> branch_name`.
- At this point, the software engineer must manually resolve the conflict: decide which parts of the code remain and which should be removed. After making the necessary edits, the file must be saved, staged (git add), and committed.

In short, branches allow safe and parallel development, merging combines work from different lines of development, and conflicts are a natural part of collaboration that every developer must learn to resolve.

EXAMPLE OF THE MERGING PROCESS

INTRODUCTION TO GIT DIFF

What is git diff?

- git diff is a command used to show changes between commits, branches, or your working directory and the index.
- **It does not change anything** — it only displays differences.
- Important note:
- The +++ and --- lines in the diff output do not indicate whether something was added or removed.
  - They simply represent the two versions being compared:
    - --- → the “old” version
    - +++ → the “new” version

INTRODUCTION TO GIT DIFF

Example:

- Here, the - and + at the start of lines indicate removal or addition. --- and +++ just label the files.

```
diff --git a/file.txt b/file.txt
--- a/file.txt
+++ b/file.txt
@@ -1,3 +1,3 @@
-Hello world
+Hello Git
```

INTRODUCTION TO GIT DIFF

Comparing staged changes: --staged (or --cached)

- By default, git diff compares your working directory with the index (staged area).
- To compare staged changes with the last commit:
- Shows changes that **are staged for the next commit**, i.e., tracked files that have been modified and added with git add.

```
git diff --staged
```

INTRODUCTION TO GIT DIFF

Comparing specific commits:

- You can compare changes between two commits using their commit IDs:
- Replace <commit1> and <commit2> with the **hashes** of the commits you want to compare.
- Example:
- Output shows **what changed from commit 3a5f1b2 to commit 9c7e8d0**.
- Hint: git log --oneline

```
git diff <commit1> <commit2>
```

```
git diff 3a5f1b2 9c7e8d0
```

INTRODUCTION TO GIT DIFF

Order of commit IDs matters:

- git diff A B ≠ git diff B A
- The first commit (A) is considered the “old” version.
- The second commit (B) is considered the “new” version.
- Lines starting with - were removed from A → B, lines starting with + were added in B compared to A.
- Example:

```
git diff A B   # shows changes to get from A to B
git diff B A   # shows changes to get from B to A (reversed)
```

SUMMARY - GIT DIFF

- git diff is a read-only comparison tool.
- +++ / --- do not indicate changes, only file versions.
- Use --staged to check what is staged for commit.
- Use commit IDs to compare specific commits.
- Order matters when comparing commits.
- git diff –staged or git diff --cached

INTRODUCTION TO GIT STASH

What is git stash?

- git stash is a command used to temporarily save changes in your working directory and index without committing them.
- It is useful when you want to switch branches or work on something else without losing your current changes.
- Think of it as a “clipboard” for your changes.
- Basic idea:

```
# Save current changes to stash
git stash

# Apply stashed changes later
git stash apply
```

INTRODUCTION TO GIT STASH

How it works?

- When you run git stash:
  - Changes in tracked files are saved to a new stash entry.
  - Working directory is reset to match HEAD (last commit).
- By default, unstaged changes are stashed.
- You can also include untracked files with -u (or --include-untracked):
- git stash -u

INTRODUCTION TO GIT STASH

Managing multiple stashes?

- Each stash is stored in a stack-like structure.
- Commands:

```
git stash list          # Show all stashes
git stash apply         # Apply the most recent stash
git stash apply stash@{2}       # Apply specific stash
git stash pop           # Apply and remove the most recent stash
git stash drop stash@{1}        # Delete specific stash
git stash clear         # Delete all stashes
```

INTRODUCTION TO GIT STASH

Important Note About the Stash Mechanism

The stash copies all changes from the branch without committing them into a single shared list.

- This means you can stash from one branch and apply it to another branch.
- Stashes are not independent per branch — there is one global stash stack.
- You need experience to handle stashes carefully, because applying or adding changes to the wrong branch can cause issues.

Without careful management, using git stash can create more problems than it solves. Avoid just doing git add . and committing from a different branch without checking your stashes first.

INTRODUCTION TO GIT STASH

Stash naming

- You can add a message for clarity:
- Helps to remember the purpose of each stash.

```
git stash save "WIP: fixing login bug"
```

INTRODUCTION TO GIT STASH

Stash and branches

- Stashes can be applied to any branch, not just the one they were created on.
- This makes them useful for:
  - Quick context switches
  - Experimenting without committing incomplete work
  - Moving work between branches

INTRODUCTION TO GIT STASH

Quick tips

- git stash = save + clean working directory.
- git stash pop = restore and remove from stash.
- git stash apply = restore without removing.
- Include untracked files: git stash -u.
- Stashes are temporary storage, not a substitute for commits.

SUMMARY - INTRODUCTION TO GIT STASH

- Use stash when you need to pause your work without committing.
- Remember: stash is a stack → LIFO (last in, first out).
- Always check git stash list to avoid losing changes.

RENAMING FILES IN GIT – IMPORTANT NOTE

When working with Git, it's important to understand how file renaming is handled. Git does not automatically detect a rename as a single action. Instead, it treats it as:

- Deletion of the old file
- Creation of a new, untracked file

So, if you rename a file manually (e.g., from oldName.txt to newName.txt), Git will see oldName.txt as deleted and newName.txt as a new file.

- To properly reflect this change in Git, you should:
- Git doesn’t track file names — it tracks content. Renaming a file is treated as removing one and adding another. Always stage both the deletion and the new file to keep your history clean and understandable.

```
$ git add oldName.txt
```

WHAT IS A GIT BRANCH?

- A branch in Git is like a separate line of development. It allows you to work on new features, bug fixes, or experiments without affecting the main codebase. The default branch is usually called main or master.
- Branches help teams collaborate safely and efficiently by isolating changes until they’re ready to be merged.
- Think of branches as parallel timelines. You can develop safely in one branch, test your changes, and merge them back when everything works. This is a core concept in modern version control.

GIT BRANCH – CREATE A NEW BRANCH

- The command git branch shows a list of all branches in your repository. The currently active branch is marked with an asterisk (*).
- When working with a repository that has multiple branches, we can switch between them using two different commands. This is because modern versions of Git introduced standardized naming conventions, but the older commands were kept to ensure backward compatibility and to avoid forcing experienced users to relearn everything from scratch.

```
git branch new_branch
```

```
$ git switch second_branch
```

```
$ git checkout second_branch
```

GIT STATUS & GIT BRANCH

Depending on the shell or terminal, Git can display additional information in the prompt, such as the current branch, whether all files are tracked, or if there are uncommitted changes. However, to check which branch you are on, you can always use the git status command or git branch without any parameters.

```
$ git branch
* master
  second
```

```
$ git status
On branch master
...
```

THANK

You

TABLE OF CONTENTS

PART 1

Foundations

SLIDE 1 — COURSE OVERVIEW & AGENDA

What you will learn:

Part	Topics
1 – Foundations	What a worktree is, repository anatomy, the context-switching problem
2 – Core Commands	`add`, `list`, `remove`, `prune`, `lock`, `move`, `repair`
3 – Bare Repositories	What they are, why to use them, full setup walkthrough
4 – Advanced Features	Detached HEAD, `--orphan`, `--no-checkout`, sparse checkout, per-worktree config, hooks, internal structure
5 – Comparisons & Gotchas	Worktrees vs. clones, edge cases, common mistakes
6 – Workflows	Standard workflow, bare-repo workflow, agentic/AI coding
Summary	Full cheat sheet

> **Prerequisites:** Basic Git knowledge (clone, commit, branch, push/pull)

SLIDE 2 — WHAT IS A GIT WORKTREE?

A **worktree** (working tree) is a directory containing a **checked-out version** of a Git project.

```
After git clone:

 project/
  ├── .git/          ← the REPOSITORY (history, branches, refs, objects…)
  ├── index.html     ┐
  ├── style.css      ├── the DEFAULT WORKTREE (checked-out project files)
  └── README.md      ┘
```

- ⚒ The **repository** (`.git/`) is Git's database — it stores everything Git knows
- ⚒ The **worktree** is where you *read and edit* files

> **Key insight:** One repository can be linked to **multiple worktrees** — multiple working directories — all sharing the same history, branches, and remotes.

SLIDE 3 — REPOSITORY ANATOMY A

```
project/
├── .git/                ← REPOSITORY (single source of truth)
│   ├── HEAD               ← currently checked-out branch/commit
│   ├── config             ← repo-level configuration
│   ├── objects/           ← all commits, file snapshots (blobs), trees
│   │   ├── pack/
│   │   └── info/
│   ├── refs/              ← branch and tag pointers
│   │   ├── heads/         ← local branches
│   │   ├── tags/
│   │   └── remotes/       ← remote-tracking branches
│   ├── hooks/             ← event scripts (pre-commit, post-push…)
│   └── worktrees/         ← metadata for each LINKED worktree
│
├── index.html           ← DEFAULT WORKTREE
├── style.css            (main branch checked out)
└── README.md
```

SLIDE 3 — REPOSITORY ANATOMY B

Concept	Contains	Location
`Repository`	All commits, branches, history, config	`.git/`
`Default worktree`	Project files at checked-out commit	Root of clone
`Linked worktree`	Project files at a *different* checked-out branch	Any directory

SLIDE 4 — THE CONTEXT-SWITCHING PROBLEM A

Scenario: You're developing `feature-A`, deep in the middle of changes.
A critical bug is reported — you must switch to `main` immediately.

```
$ git status
On branch feature-A
Changes not staged for commit:
    modified:   src/auth.js
    modified:   src/login.css
    new file:   src/oauth.js
```

SLIDE 4 — THE CONTEXT-SWITCHING PROBLEM B

What are your options?

Option	Problem
`git commit -m "WIP"`	Pollutes history with half-done commits
`git stash`	Fragile — easy to forget, pop to wrong branch, or accidentally drop
`Clone the repo again`	Wastes disk space, history duplicated, must fetch each clone separately

> None of these are great. There has to be a better way.

SLIDE 5 — WORKTREES AS THE SOLUTION A

With **multiple worktrees**, you maintain several working directories linked to the **same repository**:

```
portfolio/
├── .git/          ← single shared repository
├── main/          ← worktree: main branch (stable reference)
├── feature-a/     ← worktree: feature-A (uncommitted work stays here untouched)
└── hotfix-99/     ← worktree: hotfix-99 (fresh start, no stashing needed)
```

SLIDE 5 — WORKTREES AS THE SOLUTION B

Workflow:

```
# You were working here — uncommitted changes stay untouched
cd ~/portfolio/feature-a

# Switch context by literally switching directories
cd ~/portfolio/hotfix-99

# Work on hotfix, commit, push — feature-a is completely unaffected
git add . && git commit -m "fix: resolve auth crash" && git push

# Done — go back exactly where you left off
cd ~/portfolio/feature-a
# Your uncommitted changes are right where you left them ✅
```

SLIDE 6 — KEY RULES & PROPERTIES A

1. **Shared repository** — all worktrees share the same `.git/`, commit history, remotes, and refs
2. **One fetch updates all** — `git fetch` from *any* worktree updates the entire shared repo
3. **Branch exclusivity** — the same branch **cannot** be checked out in two worktrees simultaneously
4. **Independent index** — each worktree has its own staging area and `HEAD`

...

SLIDE 6 — KEY RULES & PROPERTIES B

5. **Independent in-flight states** — rebase, merge, cherry-pick in one worktree does NOT affect others
6. **Shared stash list** — stashes are stored in the shared repo and visible from all worktrees
7. **No repository duplication** — unlike multiple clones, history is stored only once

```
# Branch exclusivity in action:
$ git worktree add -b main ../main-copy
fatal: 'main' is already checked out at '/path/to/project'
```

PART 2

Core Commands

SLIDE 7 — ALL WORKTREE SUBCOMMANDS AT A GLANCE

```
git worktree add      # Create a new linked worktree
git worktree list     # List all worktrees (main + linked)
git worktree remove   # Delete a linked worktree
git worktree prune    # Remove stale administrative data      ★
git worktree lock     # Prevent a worktree from being pruned  ★
git worktree unlock   # Remove lock from a worktree           ★
git worktree move     # Move a worktree to a new path         ★
git worktree repair   # Fix broken worktree path references   ★
```

> ★ = commands frequently omitted in beginner tutorials

SLIDE 8 — GIT WORKTREE ADD — BASIC USAGE A

Syntax: `bash git worktree add [options] <path> [<branch-or-commit>]`

Most common patterns:

```
# 1. Check out an EXISTING branch into a new worktree
git worktree add ./main main
# 2. Shorthand — branch name inferred from the last path component
git worktree add ./hotfix-99
# 3. Create a NEW branch (from current HEAD) and check it out
git worktree add -b feature-B ../feature-b
# 4. Create a NEW branch from an explicit base branch ✅ RECOMMENDED
git worktree add -b feature-B ../feature-b main
git worktree add -b hotfix-99 ../hotfix-99 main
```

> ⚠️ Without specifying a base, Git uses **current HEAD** as the starting point.
> You may accidentally inherit unfinished commits from your current feature branch!

SLIDE 9 — GIT WORKTREE ADD — ALL FLAGS

Flag	Description	Example
`-b <branch>`	Create new branch	`git worktree add -b feat ./feat main`
`-B <branch>`	Create or force-reset existing branch	`git worktree add -B feat ./feat main`
`--detach` / `-d`	Detached HEAD (specific commit or tag)	`git worktree add -d ./review abc1234`
`--orphan <branch>`	New branch with no history	`git worktree add --orphan gh-pages ./docs`
`--no-checkout` / `-n`	Don't check out files (empty dir)	`git worktree add -n -b feat ./feat main`
`--lock [--reason <str>]`	Lock immediately after creation	`git worktree add --lock -b feat ./feat`
`--track`	Set up remote tracking explicitly	`git worktree add --track -b feat ./feat origin/feat`
`--guess-remote`	Auto-detect remote tracking from name	`git worktree add --guess-remote ./feat feat`
`-q` / `--quiet`	Suppress progress output	`git worktree add -q -b feat ./feat main`

SLIDE 10 — GIT WORKTREE LIST A

```
# Standard human-readable output
git worktree list
```

```
/home/user/portfolio              abc1234  [main]
/home/user/portfolio/feature-b    def5678  [feature-B]
/home/user/portfolio/hotfix-99    9ab0cd1  [hotfix-99]
/home/user/portfolio/old-review   1a2b3c4  (detached HEAD)
```

SLIDE 10 — GIT WORKTREE LIST B

```
# Verbose (shows lock status, prunable, etc.)
git worktree list -v
# Machine-readable output (useful in shell scripts / automation)
git worktree list --porcelain
```

```
worktree /home/user/portfolio
HEAD abc1234abc1234abc1234abc1234abc1234abc1234
branch refs/heads/main

worktree /home/user/portfolio/feature-b
HEAD def5678def5678def5678def5678def5678def5678
branch refs/heads/feature-B
locked reason: on external USB drive
```

SLIDE 11 — GIT WORKTREE REMOVE A

```
# Remove a clean worktree (no uncommitted changes)
git worktree remove ./hotfix-99

# Force-remove even if there are uncommitted changes
git worktree remove --force ./hotfix-99
git worktree remove -f ./hotfix-99

# Force-remove a LOCKED worktree (requires --force twice)
git worktree remove --force --force ./experimental
```

⚠️ `git worktree remove` deletes the **working directory** and `.git/worktrees/<name>/`

The branch itself is NOT deleted.

SLIDE 11 — GIT WORKTREE REMOVE B

```
# After removing the worktree, delete the branch if no longer needed:
git branch -d hotfix-99       # safe delete (fails if not merged)
git branch -D hotfix-99       # force delete
```

⚠️ **The main (default) worktree cannot be removed with this command.**

SLIDE 12 — GIT WORKTREE PRUNE ⭐A

What is stale data? When a worktree directory is deleted **manually** via the filesystem (e.g. `rm -rf ./hotfix-99`), Git's metadata in `.git/worktrees/` becomes stale.

```
# Clean up all stale worktree references
git worktree prune

# Dry run — preview without making changes
git worktree prune --dry-run
git worktree prune -n

# Verbose output
git worktree prune --verbose

# Control the expiry window (default: 3 months)
git worktree prune --expire="3 months ago"
git worktree prune --expire=now
```

SLIDE 12 — GIT WORKTREE PRUNE ★B

When to run manually:

- ⚒ After accidentally deleting a worktree folder with `rm -rf`
- ⚒ After a disk or filesystem failure
- ⚒ When `git worktree list` shows paths that no longer exist
- ⚒ As part of regular housekeeping alongside `git gc`

SLIDE 13 — GIT WORKTREE LOCK / UNLOCK⭐A

```
# Lock an existing worktree
git worktree lock ./experimental

# Lock with a reason message
git worktree lock --reason "stored on USB SSD, mount when needed" ./experimental

# Create and lock immediately
git worktree add --lock --reason "external drive" -b experiment ./experimental main

# Unlock
git worktree unlock ./experimental
```

SLIDE 13 — GIT WORKTREE LOCK / UNLOCK ⭐B

Removal behavior for locked worktrees:

```
git worktree remove ./experimental
# error: cannot remove a locked working tree

git worktree remove --force ./experimental
# error: use 'remove -f -f' to override

git worktree remove --force --force ./experimental
# ✅ removed (double force required)
```

SLIDE 14 — GIT WORKTREE MOVE ⭐A

```
# Move safely
git worktree move ./feature-b ./archive/feature-b

# Move a locked worktree
git worktree move --force ./experimental /mnt/usb/experimental
```

Why NOT just use `mv`?

```
# ❌ WRONG:
mv ./feature-b ./archive/feature-b
# Leaves .git/worktrees/feature-b/gitdir pointing to OLD path
# Git considers worktree stale → may prune it
# Cannot run git commands inside it
```

SLIDE 14 — GIT WORKTREE MOVE ★B

What `git worktree move` does:

1. Moves the directory on the filesystem
2. Updates `.git/worktrees/<name>/gitdir` to new location
3. Updates the `.git` file inside the worktree

SLIDE 15 — GIT WORKTREE REPAIR ⭐ A

Scenario A: Worktree moved manually

```
cd /new/location/of/feature-b
git worktree repair
```

Scenario B: Main repository moved

```
git worktree repair ./feature-b ./hotfix-99
```

SLIDE 15 — GIT WORKTREE REPAIR⭐B

Scenario	What Broke	Where to Run
`Worktree moved manually`	`gitdir` file has stale path	Inside moved worktree
`Main repo moved`	`.git` pointer in each worktree is stale	From main repo, passing paths

PART 3

Bare Repositories

SLIDE 16 — WHAT IS A BARE REPOSITORY? A

Standard clone:

```
portfolio/
├── .git/          ← repository data
├── index.html     ←┐
├── style.css      ├── DEFAULT WORKTREE
└── README.md      ←┘
```

Bare clone:

```
portfolio.git/     ← repository data ONLY (no working tree)
├── HEAD
├── config         ← core.bare = true
├── objects/
└── refs/
```

SLIDE 16 — WHAT IS A BARE REPOSITORY? B

```
# Verify if a repo is bare:
git config --get core.bare
# true  → bare repo
# false → standard repo
```

SLIDE 17 — STANDARD CLONE VS. BARE CLONE

```
# STANDARD CLONE
git clone https://github.com/user/repo.git
# repo/
# ├── .git/
# ├── index.html
# └── ...
# BARE CLONE
git clone --bare https://github.com/user/repo.git
# repo.git/   ← no working tree
# ├── HEAD
# ├── config  ← core.bare = true
# └── ...
# BARE INTO .git/ FOLDER (preferred for worktree workflows)
mkdir portfolio && cd portfolio
git clone --bare https://github.com/user/repo.git .git
# portfolio/
# └── .git/   ← bare repo lives cleanly here
```

SLIDE 18 — WHY USE A BARE REPO WITH WORKTREES? A

Standard clone + worktrees — messy:

```
home/
├── portfolio/          ← default worktree (tempts working on main)
│   └── .git/
├── feature-b/          ← outside project folder
└── hotfix-99/          ← outside project folder
```

Bare repo + worktrees — clean:

```
portfolio/              ← single parent for EVERYTHING
├── .git/               ← bare repo
├── main/               ← worktree: main (reference only)
├── feature-b/          ← worktree: feature-B
├── hotfix-99/          ← worktree: hotfix-99
└── experiment/         ← worktree: experimental
```

SLIDE 18 — WHY USE A BARE REPO WITH WORKTREES? B

- ✅ All worktrees inside one bounded directory
- ✅ No default worktree to accidentally pollute
- ✅ One `ls` shows entire project state

SLIDE 19 — CLONING A BARE REPOSITORY — FULL WALKTHROUGH

```
# 1. Create parent folder
mkdir portfolio && cd portfolio
# 2. Clone as bare into .git/
git clone --bare https://github.com/user/repo.git .git
# 3. Verify — only .git/ exists
ls -a
# .git/
# 4. No working tree yet:
git status
# fatal: this operation must be run in a work tree
# 5. Explore bare repo contents
ls .git/
# HEAD  config  description  hooks/  info/  objects/  packed-refs  refs/
# 6. Confirm bare
git config --get core.bare
# true
```

SLIDE 20 — BARE REPO — FULL WORKFLOW EXAMPLE

```
# Step 1: main worktree as reference
git worktree add ./main main
# Step 2: new feature
git worktree add -b feature-login ./feature-login main
cd ./feature-login
git add . && git commit -m "feat: add JWT login"
git push origin feature-login
# Step 3: urgent hotfix — no stashing needed
cd ..
git worktree add -b hotfix-csrf ./hotfix-csrf main
cd ./hotfix-csrf
git add . && git commit -m "fix: prevent CSRF"
git push origin hotfix-csrf
# Step 4: update main reference after merge
cd ../main
git pull
# Step 5: clean up
cd ..
git worktree remove ./hotfix-csrf
git branch -d hotfix-csrf
# Step 6: resume feature — nothing lost ✅
cd ./feature-login
```

PART 4

Advanced Features

SLIDE 21 — DETACHED HEAD IN WORKTREES ★

```
# Specific commit SHA
git worktree add --detach ./review-v1 a1b2c3d4
git worktree add -d ./review-v1 a1b2c3d4

# Version tag
git worktree add -d ./review-v2.0 v2.0.0

# 3 commits before HEAD
git worktree add -d ./review-previous HEAD~3
```

```
/path/portfolio/main          abc1234  [main]
/path/portfolio/review-v2.0   def5678  (detached HEAD)
```

Use cases:

- ⚒ Review old code without affecting any branch

SLIDE 22 — --ORPHAN FLAG ⭐

```
git worktree add --orphan gh-pages ./docs-site

cd ./docs-site
# On branch gh-pages — No commits yet

echo "<!DOCTYPE html><html><body>Docs</body></html>" > index.html
git add .
git commit -m "initial: gh-pages setup"
git push origin gh-pages
```

Use cases:

- ⚒ GitHub Pages (`gh-pages`) — no relation to source code
- ⚒ `wiki` branch separate from application code
- ⚒ Legal / license content maintained independently

SLIDE 23 — --NO-CHECKOUT ★

```
# Empty worktree directory
git worktree add -n -b feature-sparse ./sparse-wt main

# With sparse checkout:
cd ./sparse-wt
git sparse-checkout init --cone
git sparse-checkout set src/ tests/
git checkout

ls
# src/   tests/
```

Use cases:

- ⚒ CI/CD pipelines needing only specific files
- ⚒ Large monorepos — each worktree shows only relevant subset

SLIDE 24 — SPARSE CHECKOUT + WORKTREES ★

```
git clone --bare https://github.com/company/monorepo.git .git

# Worktree 1: frontend only
git worktree add -n -b feat-ui ./feat-ui main
cd ./feat-ui
git sparse-checkout init --cone
git sparse-checkout set frontend/ shared/
git checkout

# Worktree 2: backend only
cd ..
git worktree add -n -b feat-api ./feat-api main
cd ./feat-api
git sparse-checkout init --cone
git sparse-checkout set backend/ shared/
git checkout

# Worktree 3: full reference
cd ..
```

SLIDE 25 — REMOTE TRACKING & --GUESS-REMOTE ★

```
# Explicit tracking
git worktree add --track -b feature-x ./feature-x origin/feature-x

# Auto-detect tracking
git worktree add --guess-remote ./feature-x feature-x

# Set as global default
git config --global worktree.guessRemote true
```

SLIDE 26 — PER-WORKTREE CONFIGURATION ★ (GIT 2.34+)

```
git config extensions.worktreeConfig true

# Work identity
cd ./work-project
git config --worktree user.email "alice@company.com"
git config --worktree user.signingkey "WORK_GPG_KEY"

# Personal identity
cd ../personal-fork
git config --worktree user.email "alice@personal.com"
git config --worktree user.signingkey "PERSONAL_GPG_KEY"
```

```
.git/
├── config                ← shared (all worktrees)
├── config.worktree       ← main worktree overrides
└── worktrees/
```

SLIDE 27 — INTERNAL STRUCTURE: .GIT/WORKTREES/ ★

```
.git/
└── worktrees/
    └── feature-b/
        ├── gitdir          ← path to .git FILE inside worktree
        ├── commondir       ← path back to main .git/
        ├── HEAD            ← checked-out branch here
        ├── index           ← independent staging area
        ├── ORIG_HEAD
        ├── MERGE_HEAD      ← only during merge
        ├── REBASE_HEAD     ← only during rebase
        ├── locked          ← only if locked
        └── config.worktree ← only if worktreeConfig enabled
```

```
cat ~/portfolio/feature-b/.git
# gitdir: /home/user/portfolio/.git/worktrees/feature-b
```

SLIDE 28 — HOOKS IN WORKTREES ⭐

```
.git/hooks/
├── pre-commit    ← runs for ALL worktrees
├── post-commit   ← runs for ALL worktrees
├── pre-push      ← runs for ALL worktrees
└── post-merge    ← runs for ALL worktrees
```

```
#!/bin/bash

# ❌ WRONG — hardcoded path:
cd /home/user/portfolio && npm test

# ✅ CORRECT — dynamic resolution:
WORK_TREE=$(git rev-parse --show-toplevel)
cd "$WORK_TREE" && npm test

# ✅ For main .git/ path:
```

Part 5 — Comparisons & Gotchas

SLIDE 29 — WORKTREES VS. MULTIPLE CLONES

Feature	Worktrees	Multiple Clones
`Disk usage`	✅ History stored once	❌ Full `.git/` duplicated
`git fetch`	✅ One fetch updates all	❌ Fetch each separately
`Branch visibility`	✅ All branches visible	⚠️ Only fetched branches
`Branch exclusivity`	✅ Git prevents duplicates	❌ Same branch in two clones
`Stash sharing`	✅ One shared stash list	❌ Separate per clone

SLIDE 30 — EDGE CASES & COMMON GOTCHAS ⭐

```
# 1. Stashes are repo-wide — name them explicitly
git stash push -m "feature-b: WIP auth refactor"
git stash push -m "hotfix: WIP rate limit fix"

# 2. Cannot remove main worktree
git worktree remove .
# fatal: 'remove' is not allowed for a main worktree

# 3. HEAD~ is relative to each worktree's own HEAD
# feature-b: HEAD~1 = commit before feature-b tip
# hotfix-99: HEAD~1 = commit before hotfix-99 tip

# 4. Always prune after manual deletions
git worktree list     # stale entry showing?
git worktree prune    # clean it up

# 5. Branch persists after worktree removal
git worktree remove ./hotfix-99
```

Part 6 — Workflows & Use Cases

SLIDE 31 — AGENTIC / AI CODING WORKFLOWS ⭐

```
# Parallel agent sessions
git worktree add -b feat-login ./feat-login main
git worktree add -b feat-dashboard ./feat-dashboard main

# Terminal 1
cd ./feat-login
claude "Implement JWT login with refresh tokens"

# Terminal 2 — simultaneously
cd ./feat-dashboard
claude "Implement analytics dashboard with Chart.js"

# Zero interference ✅
```

Custom slash command in Claude Code:

SLIDE 32 — BEST PRACTICES

```
# 1. Always explicit base branch
git worktree add -b feature-x ./feature-x main   # ✅
git worktree add -b feature-x ./feature-x        # ⚠ risky

# 2. Mirror branch names in folder names
git worktree add -b hotfix-auth ./hotfix-auth main

# 3. Navigate to parent before creating
cd ~/portfolio
git worktree add -b feat-x ./feat-x main

# 4. Clean up consistently
git worktree remove ./hotfix-auth
git branch -d hotfix-auth
git worktree prune

# 5. Lock removable media worktrees
git worktree lock --reason "on external SSD" ./experiment

# 6. Enable per-worktree config for multiple identities
git config extensions.worktreeConfig true
```

Quick Reference Cheat Sheet

```
# SETUP
git clone --bare <url> .git

# ADD WORKTREE
git worktree add ./path                    # existing branch
git worktree add ./path <branch>           # existing branch explicit
git worktree add -b <new> ./path main      # new branch from main ✅
git worktree add -B <branch> ./path main   # create or reset
git worktree add -d ./path <commit-or-tag> # detached HEAD
git worktree add --orphan <branch> ./path  # orphan branch
git worktree add -n -b <branch> ./path main  # no checkout
git worktree add --lock -b <branch> ./path main  # create and lock
git worktree add --track -b <b> ./path o/<b>     # with remote tracking
git worktree add --guess-remote ./path <branch>  # auto-detect tracking

# LIST
git worktree list                  # human-readable
git worktree list -v               # verbose
git worktree list --porcelain      # for scripts

# REMOVE
git worktree remove ./path         # clean
git worktree remove -f ./path      # force
git worktree remove -f -f ./path   # force locked

# PRUNE
git worktree prune                 # remove stale
git worktree prune -n              # dry run
git worktree prune --expire=now    # immediate

# LOCK / UNLOCK
git worktree lock --reason "msg" ./path
git worktree unlock ./path

# MOVE / REPAIR
git worktree move ./old ./new
git worktree repair ./path

# CONFIGURATION
git config extensions.worktreeConfig true
git config --worktree user.email "x@x.com"
git config --global worktree.guessRemote true
git config --global push.autoSetupRemote true
```